

1. Introduction & Literature Review

Music recommendation systems are used by many streaming platforms to help users find songs they may enjoy. Apps like Spotify, Apple Music, and YouTube Music use machine learning to recommend songs based on listening history, user behavior, and song features. This is important because users often want music that matches their mood, activity, or personal taste.

Machine learning can help with this problem because it can learn patterns from song data. For example, features like energy, danceability, tempo, and valence can help describe the “vibe” of a song. These features can be used to predict whether a user may like a song.

Researchers have studied music recommendation systems using methods such as content-based filtering, collaborative filtering, and hybrid recommendation systems. Content-based filtering focuses on song features, while collaborative filtering uses behavior from other users. Companies such as Spotify also use machine learning to personalize playlists and recommendations.

One limitation is that music taste is very personal and can be hard to model. Another limitation is that some datasets do not include real user likes, skips, or playlist saves. Because of this, this project uses song features to create a simple AI DJ model that predicts whether a song matches a user’s preferred vibe.

2. Problem Definition

The goal of this project is to use machine learning to predict whether a user would like a song.

This project focuses on building a simple AI DJ recommendation system based on song vibe.

The input features include music information such as energy, danceability, valence, tempo, loudness, acousticness, instrumentalness, liveness, and other song features. The output is a binary label where 1 means the song is liked and 0 means the song is not liked.

This is a supervised machine learning classification problem because the model is trained using labeled examples. Since the dataset did not include real user like or dislike data, a simulated “liked” column was created using energy, danceability, and valence.

3. Approach, Materials, and Methods

This project used the Music Info.csv file from the Million Song Dataset + Spotify + Last.fm dataset. The dataset contains song information such as song title, artist, year, duration, and audio features. The dataset included 50,683 songs and 21 columns before cleaning.

The dataset was cleaned before training the models. Columns such as track ID, Spotify preview URL, Spotify ID, genre, and tags were removed because they were not needed for model training. The genre and tags columns also had many missing values, so they were not used.

A new target column called “liked” was created. Songs with higher energy, danceability, and valence were labeled as liked songs. In this project:

Label Meaning

1 Liked

0 Not Liked

The data was split into 80% training data and 20% testing data. Four machine learning models were tested:

Model

Logistic Regression

Decision Tree

Random Forest

K-Nearest Neighbors (KNN)

Logistic Regression was used as a simple starting model. Decision Tree and Random Forest were used because they can handle more complex patterns in the data. KNN was used to compare a distance-based model with the other models.

The project was completed in Python using Google Colab. The main libraries used were pandas, numpy, scikit-learn, and matplotlib.

4. Results

Four machine learning models were tested and compared using accuracy.

Model	Accuracy
Random Forest	100.00%
Decision Tree	100.00%
Logistic Regression	87.58%
KNN	70.67%

Random Forest and Decision Tree both had the highest accuracy. Random Forest was selected as the final model because it is usually more stable than a single decision tree and is less likely to overfit in real-world use.

For the Random Forest model, the detailed classification results were:

Metric	Not Liked (0)	Liked (1)
Precision	1.00	1.00
Recall	1.00	1.00
F1-Score	1.00	1.00

Logistic Regression performed lower than Random Forest and Decision Tree. Its classification report showed:

Metric	Not Liked (0)	Liked (1)
Precision	0.90	0.78
Recall	0.94	0.68
F1-Score	0.92	0.73

KNN had the lowest accuracy at 70.67%. This means it was not as strong for this dataset compared to the other models.

The model performance improved as more advanced models were tested. Logistic Regression was the baseline model, KNN performed the weakest, and Random Forest and Decision Tree gave the best results.

5. Discussion

The results show that machine learning can be used to build a simple AI DJ recommendation system. Random Forest and Decision Tree performed the best, with both reaching 100% accuracy. Random Forest was chosen as the final model because it combines many decision trees, which usually makes it more reliable than using one decision tree by itself.

Logistic Regression did not perform as well because music features can have more complex relationships. KNN performed the weakest, possibly because it depends on distance between data points and may not work as well when many song features are used together.

One challenge in this project was that the dataset did not include real user likes or skips. To solve this, I created a simulated “liked” column using energy, danceability, and valence. This helped

make the project possible, but it is also a limitation. The model is learning a rule-based version of user preference instead of real listener behavior.

Another limitation is that the “liked” label was created from the same features used to train the model. This helped the model perform very well, but it may not fully represent how users choose music. A real AI DJ would need real user behavior data such as likes, skips, playlist saves, and replay history.

These results are like what music platforms are doing at a larger scale. In real music platforms, machine learning is often used to personalize playlists and suggest songs based on listening behavior and song characteristics. This project is a smaller version of that idea.

6. Conclusion

This project showed that machine learning can be used to build a simple AI DJ recommendation system. Four models were tested, and Random Forest was selected as the final model.

I learned how to clean a music dataset, create a target column, train multiple models, compare results, and generate song recommendations. I also learned that the quality of the target label is very important because it affects what the model learns.

In the future, this project could be improved by using real user listening data, such as likes, skips, playlist saves, and replay history. It could also be improved by using more advanced recommendation methods, such as collaborative filtering, clustering, or deep learning.